Feature Preprocessing, PCA, and Eigendecomposition

Comprehensive Theoretical and Practical Guide for Master's Defense

Document Purpose & Thesis Context

This technical reference document provides an exhaustive, mathematically rigorous, and intuitive walkthrough of:

- **Pipeline Technologies:** OpenCV decoding, Float16 + SDPA in V-JEPA 2, and 4-bit NF4 Quantization in Qwen2.5-VL.
- **Frozen-Feature Protocol:** What it means, why it is used, and exact pipeline order.
- **Feature Standardization (StandardScaler):** Formulae, variance distortion prevention, and data leakage safeguards.
- **Linear Algebra Foundations:** Eigenvectors, eigenvalues, characteristic equations, and spectral decomposition of symmetric matrices.
- **Principal Component Analysis (PCA):** Step-by-step mathematical derivation, variance maximization, 95% variance retention rule, and orthogonal projection.
- **Step-by-Step Worked Numerical Example:** Complete hand calculation showing standardization, covariance matrix, eigenvalue solving, eigenvector normalization, and 1D projection.
- **Architectural Rationale:** Why baseline classifiers require PCA while HAF-STD Net bypasses it.

Thesis Focus:	Action Recognition via Self-Supervised & Multimodal Foundational Models
Foundation Models:	V-JEPA 2.1 (ViT-H, 654M) & Qwen2.5-VL (7B, 4-bit NF4)
Datasets:	FineGym (Sub-action Gymnastics) & HMDB51 (51 Human Actions)
Novel Architecture:	HAF-STD Net (4.8M Trainable Parameters)

October 8, 2026

Contents

1 End-to-End Methodology Timeline & Key Concepts **2**
1.1 Why OpenCV and How it Works . . . 2
1.2 Why Float16 and SDPA in V-JEPA 2 . . . 3
1.3 What is 4-bit NF4 Quantization in Qwen2.5-VL? . . . 4
1.4 What are Frozen Features and When Does Freezing Occur? . . . 5

2 Feature Preprocessing: StandardScaler **6**
2.1 Why Feature Preprocessing is Mandatory . . . 6
2.2 Mathematical Formulation . . . 6
2.3 Data Leakage: Why Fit on Train Split Only? . . . 6

3 Linear Algebra Foundations: Eigenvectors and Eigenvalues **8**
3.1 Matrix Multiplication as Geometric Transformation . . . 8
3.2 The Fundamental Eigen-Equation . . . 8
3.3 How to Find Eigenvalues: The Characteristic Equation . . . 9
3.4 Why the Covariance Matrix Guarantees Perfect Properties . . . 9

4 Principal Component Analysis (PCA) Step-by-Step **11**
4.1 The Core Optimization Goal of PCA . . . 11
4.2 The Complete 5-Step PCA Algorithm . . . 12
4.3 The 95% Variance Retention Rule (Eq. 3.20) . . . 12
4.4 Dimensionality Reduction Results in the Thesis . . . 13

5 Step-by-Step Worked Numerical Walkthrough **14**
5.1 Initial Data Matrix . . . 14
5.2 Step 1: Compute Means and Standard Deviations . . . 14
5.3 Step 2: Construct Standardized Matrix $\tilde{\mathbf{Z}}$ . . . 14
5.4 Step 3: Compute the Covariance Matrix $\mathbf{C}$ . . . 15
5.5 Step 4: Solve the Characteristic Equation for Eigenvalues . . . 15
5.6 Step 5: Solve for the Eigenvectors . . . 16
5.7 Step 6: Apply 95% Cutoff and Project Data . . . 16

6 Architectural Rationale: Why Baseline Classifiers Use PCA but HAF-STD Net Bypasses It **18**
6.1 Why Baseline Classifiers Strictly Require PCA . . . 18
6.2 Why HAF-STD Net Deliberately Bypasses PCA . . . 19

7 Quick Reference & Examination Defense Cheat Sheet **20**
7.1 Top 5 Defense Questions & Bulletproof Answers . . . 20

1 End-to-End Methodology Timeline & Key Concepts

To understand where feature preprocessing and PCA fit, we must first map the entire sequential pipeline of the thesis.

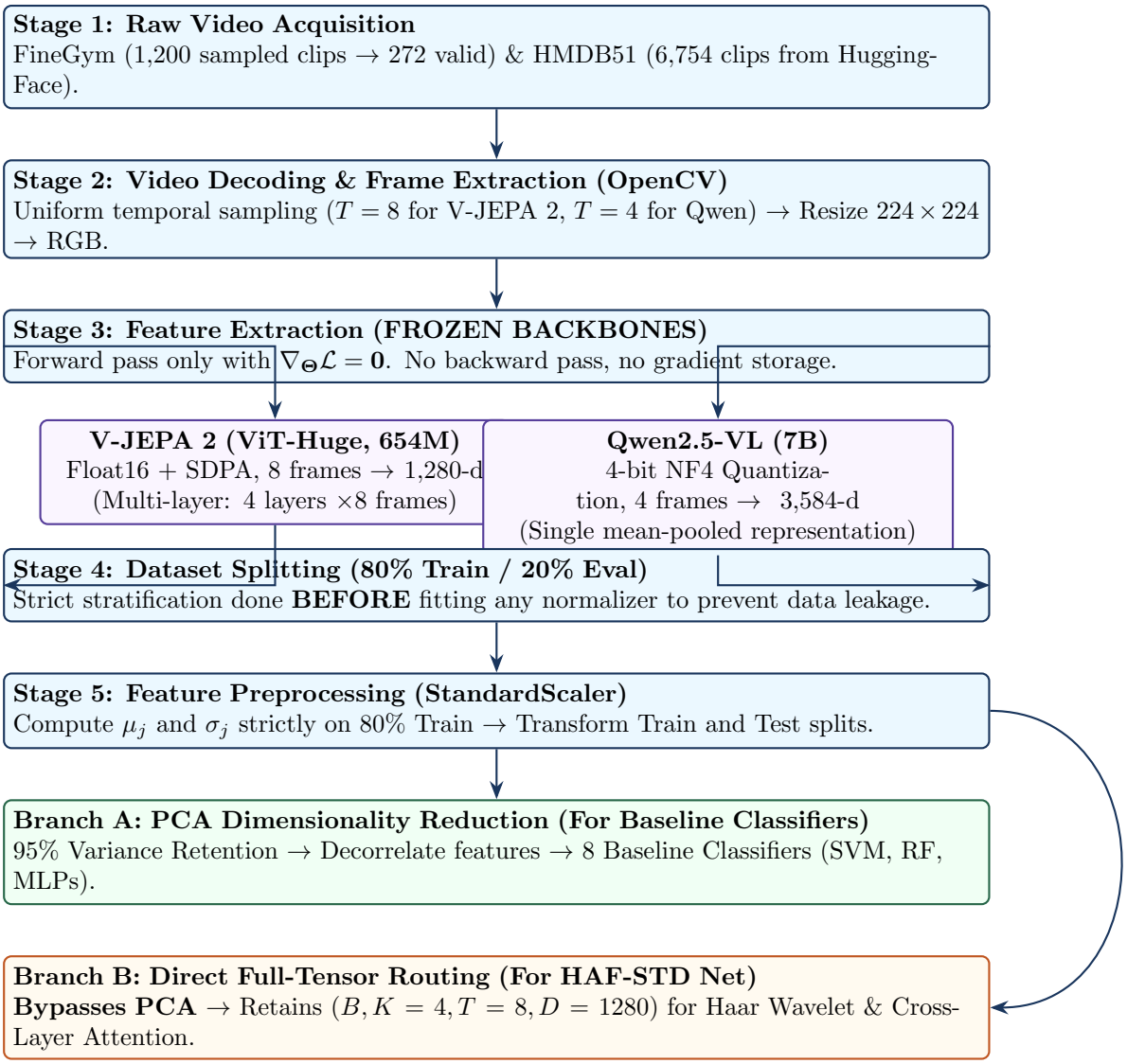


Figure 1: Sequential timeline of the thesis methodology, showing where frozen extraction, data splitting, standardization, PCA, and classifier training occur.

1.1 Why OpenCV and How it Works

Technical Mechanics of OpenCV `VideoCapture`

Why OpenCV? Raw video files are not arrays of pixels; they are compressed binary containers (e.g., MP4, AVI, MKV) encoding temporal bitstreams (such as H.264/AVC, H.265/HEVC, or VP9). Deep neural networks cannot consume MP4 files directly; they require normalized tensor representations:

$$\mathbf{X} \in \mathbb{R}^{T \times 3 \times H \times W}$$

OpenCV (Open Source Computer Vision Library) is chosen for its fast C++ backend, hardware-accelerated FFmpeg bindings, and robust demuxing capabilities.
How `VideoCapture` Works Under the Hood:

1. **Container Demuxing:** OpenCV reads container headers (e.g., MP4 atom box) to parse stream metadata (duration, container framerate FPS, total frame count N_{total}, aspect ratio).
2. **Bitstream Decoding:** It feeds compressed I-frames (Intra, keyframes), P-frames (Predicted), and B-frames (Bi-directional) into an FFmpeg codec decoder, decompressing Macroblocks/Transform blocks into uncompressed planar YUV420 pixel matrices.
3. **Color Conversion:** Raw video frames are decoded into BGR (Blue-Green-Red) byte buffers in host RAM ($H \times W \times 3$, unsigned 8-bit integers $\in [0, 255]$). OpenCV performs colorspace conversion from BGR to RGB:
$$\text{RGB}_{i,j} = \text{BGR}_{i,j}[2, 1, 0]$$
4. **Uniform Temporal Subsampling:** To capture the full motion progression of an action duration without processing thousands of redundant 30 fps frames, uniform time indexing selects T equidistant timestamps:
$$t_k = \left\lfloor k \cdot \frac{N_{\text{total}} - 1}{T - 1} \right\rfloor, \quad k \in \{0, 1, \ldots, T-1\}$$
where $T = 8$ for V-JEPA 2 and $T = 4$ for Qwen2.5-VL.
5. **Bicubic Spatial Resizing:** Each sampled frame is interpolated to the canonical resolution 224×224 pixels required by the Vision Transformer patch embedding projection layer.

1.2 Why Float16 and SDPA in V-JEPA 2

V-JEPA 2 Computational Optimizations

V-JEPA 2 uses a massive **Vision Transformer Huge (ViT-H)** architecture containing **654 million parameters** across 32 transformer blocks, with an internal hidden dimension of $D = 1{,}280$.
1. What is Float16 (Half Precision)? Standard single-precision floating point (FP32) uses 32 bits per weight: 1 sign bit, 8 exponent bits, and 23 mantissa bits (4 bytes). Float16 (FP16) uses 16 bits: 1 sign bit, 5 exponent bits, and 10 mantissa bits (2 bytes).

- **VRAM Reduction:** Storing 654M weights in FP32 requires ≈ 2.62 GB, plus activations. In FP16, weight footprint drops by 50% to ≈ 1.31 GB.
- **Tensor Core Throughput:** Modern GPUs feature dedicated Tensor Cores that execute FP16 matrix multiplications at $4\times$ to $8\times$ the FLOPS rate of standard FP32 units.

2. What is SDPA (Scaled Dot-Product Attention)? Standard attention computed in naive PyTorch:
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

requires materializing the intermediate attention affinity matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{B\times H\times N_{\text{tokens}}\times N_{\text{tokens}}}$ into GPU High-Bandwidth Memory (HBM). For high-resolution video tokens ($N_{\text{tokens}} = 8$ frames $\times\ 14 \times 14 = 1{,}568$ tokens per clip), materializing $\mathbf{S}$ across 16 attention heads incurs $O(N^2)$ memory allocations and massive memory bandwidth bottlenecks.
`torch.nn.functional.scaled_dot_product_attention` (SDPA) implements **FlashAttention-2**:

- It tiles queries, keys, and values into small chunks that fit entirely within ultra-fast GPU on-chip SRAM cache (20 TB/s bandwidth vs 1 TB/s HBM).
- It calculates softmax online using running max and sum accumulators without ever writing the large $1{,}568 \times 1{,}568$ attention matrix back to global VRAM.
- This achieves a $3\times - 5\times$ speedup and reduces attention memory complexity from $O(N^2)$ to $O(N)$.

1.3 What is 4-bit NF4 Quantization in Qwen2.5-VL?

Quantization Mechanics: NormalFloat4 (NF4)

The Problem: Qwen2.5-VL is a 7-Billion parameter Multimodal Large Language Model. Storing 7B parameters in 16-bit precision requires:

$$\text{RAM} = 7 \times 10^9 \times 2 \text{ bytes} \approx 14.0 \text{ GB}$$

Running inference with 14 GB of weights plus dynamic KV-caches and visual patch embeddings would exceed standard single-GPU VRAM limits (e.g., 16 GB GPUs) or leave zero memory for feature tensors.
What is NF4? NormalFloat4 is an information-theoretically optimal non-linear 4-bit data type developed by Dettmers et al. (QLoRA):

- Pretrained deep neural network weights follow a zero-mean normal distribution: $\mathbf{W} \sim \mathcal{N}(0, \sigma^2)$.
- Standard linear integer quantization (INT4) divides the interval $[-1, 1]$ into 16 evenly spaced bins. However, normally distributed weights have high density near 0 and very few values in the tails. Uniform bins waste precision on empty tail regions and cause high quantization error near zero.
- NF4 assigns exact quantiles of the theoretical normal distribution $q_i = \Phi^{-1}(p_i)$ to each of the 16 available 4-bit codes ($2^4 = 16$ discrete values). Every quantization bin has equal empirical probability of containing weights!
- **Result:** Qwen2.5-VL's 14 GB footprint shrinks to ≈ 3.9 GB in VRAM, allowing the 7B model to run frozen feature extraction smoothly with near-zero loss in representational accuracy.

1.4 What are Frozen Features and When Does Freezing Occur?

Concept Definition: Frozen-Feature Protocol

What does "Frozen" mean? A model is "frozen" when all its internal learned parameters (weights $\mathbf{W}$ and biases $\mathbf{b}$) are set to read-only during downstream evaluation:

$$\boldsymbol{\Theta} = \boldsymbol{\Theta}_{\text{pretrained}}, \quad \text{requires_grad} = \text{False}, \quad \nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0}$$

When video frames are passed through the network, PyTorch evaluates the model in `with torch.no_grad():` mode. No activation computation graph is saved, and no backpropagation gradients are propagated into the backbone.

At what step does it occur? Freezing is active during **Stage 3 (Feature Extraction)**. The model acts strictly as a static mathematical transformation:

$$f : \mathbb{R}^{T \times 3 \times 224 \times 224} \longrightarrow \mathbb{R}^{D}$$

Once all video clips are processed through the backbone, the outputs are stored as fixed numerical arrays on disk. The foundation backbones (V-JEPA 2 and Qwen2.5-VL) are then completely unloaded from GPU memory!

Timeline Clarification: Feature preprocessing (StandardScaler and PCA) happens **AFTER** feature extraction, operating purely on the saved feature arrays $\mathbf{Z} \in \mathbb{R}^{N \times D}$.

2 Feature Preprocessing: StandardScaler

2.1 Why Feature Preprocessing is Mandatory

Raw feature vectors extracted from deep neural networks have dimensions with vastly differing numerical magnitudes:

- Certain latent dimensions might vary between $[-0.01, +0.01]$, while other dimensions (especially after unnormalized layers or residual sums) might range from $[-15.0, +25.0]$.
- If unstandardized features are passed directly to PCA, PCA's variance maximization will mistake dimension 2 as being "2,500 times more informative" than dimension 1, purely due to arbitrary unit scaling!
- Furthermore, distance-based classifiers (SVM with RBF kernel, KNN) compute Euclidean distances $\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \sum_k (x_{ik} - x_{jk})^2$. A feature with scale 20 will completely dominate a feature with scale 0.01, effectively blinding the classifier to the smaller feature.

2.2 Mathematical Formulation

Let the extracted feature matrix from the training set be $\mathbf{Z}_{\text{train}} \in \mathbb{R}^{N_{\text{train}} \times D}$, where N_{train} is the number of video training samples and D is the embedding dimension ($D = 1{,}280$ for V-JEPA 2, $D = 3{,}584$ for Qwen2.5-VL).

StandardScaler Equations (Eq. 3.18 in Thesis)

For each feature dimension $j \in \{1, 2, \ldots, D\}$:

$$\mu_j = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} z_{ij} \tag{1}$$

$$\sigma_j = \sqrt{\frac{1}{N_{\text{train}} - 1} \sum_{i=1}^{N_{\text{train}}} (z_{ij} - \mu_j)^2} \tag{2}$$

$$\tilde{z}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j} \tag{3}$$

where:

- μ_j is the sample mean of feature dimension j.
- σ_j is the sample standard deviation (using Bessel's correction $N_{\text{train}} - 1$).
- $\tilde{z}_{ij}$ is the standardized z-score coordinate, ensuring $\mathbb{E}[\tilde{\mathbf{z}}_j] = 0$ and $\text{Var}(\tilde{\mathbf{z}}_j) = 1$.

2.3 Data Leakage: Why Fit on Train Split Only?

Critical Scientific Guardrail: Preventing Data Leakage

A catastrophic mistake in machine learning evaluation is standardizing the dataset prior to train/test splitting:

$$\mu_{\text{leaked}} = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} z_{ij} \quad \textbf{[INCORRECT / DATA LEAKAGE!]}$$

If μ and σ are computed across all N_{total} samples (training + test):

1. Information from the test distribution leaks into the training pipeline.
2. The model is indirectly exposed to the mean, variance, and extreme values of unseen test samples.
3. This yields artificially inflated test accuracy that collapses in real-world deployment.

Correct Protocol Used in Thesis:

$$\mu_j = \text{mean}(\mathbf{Z}_{\text{train}}[:, j]), \quad \sigma_j = \text{std}(\mathbf{Z}_{\text{train}}[:, j])$$

$$\tilde{\mathbf{Z}}_{\text{train}} = \frac{\mathbf{Z}_{\text{train}} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \qquad \tilde{\mathbf{Z}}_{\text{test}} = \frac{\mathbf{Z}_{\text{test}} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

The test set is transformed strictly using the training statistics $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$. The test set is never inspected during preprocessing parameter estimation.

3 Linear Algebra Foundations: Eigenvectors and Eigenvalues

Before delving into PCA, we must establish complete clarity on eigenvectors and eigenvalues. Examiners frequently probe candidates on these geometric fundamentals.

3.1 Matrix Multiplication as Geometric Transformation

A matrix $\mathbf{A} \in \mathbb{R}^{D \times D}$ acts as a linear transformation operator on vectors in $\mathbb{R}^D$. When we multiply $\mathbf{Ax}$:

- In general, the resulting vector $\mathbf{y} = \mathbf{Ax}$ has both a **different length** and points in a **different direction** (it is rotated and stretched).

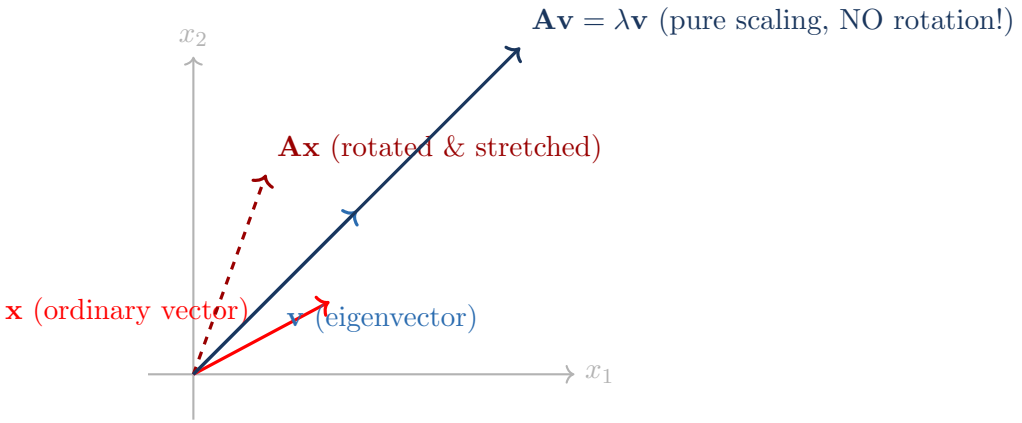


Figure 2: Geometric distinction between an arbitrary vector (rotated by $\mathbf{A}$) and an eigenvector (direction invariant, scaled by λ).

3.2 The Fundamental Eigen-Equation

The Eigenvalue Equation

A non-zero vector $\mathbf{v} \in \mathbb{R}^D$ is an **eigenvector** of a square matrix $\mathbf{A} \in \mathbb{R}^{D \times D}$ if and only if:

$$\mathbf{Av} = \lambda \mathbf{v} \tag{4}$$

for some scalar $\lambda \in \mathbb{R}$, known as the corresponding **eigenvalue**.

Intuitive Meaning of the Equation

- **Direction Invariance:** The transformation $\mathbf{A}$ does *not* rotate $\mathbf{v}$. The output $\mathbf{Av}$ lies on the exact same span (line) as $\mathbf{v}$.
- **The Eigenvalue λ:** Represents the *scaling factor* along that invariant direction:
  - If $\lambda > 1$: $\mathbf{v}$ is stretched.
  - If $0 < \lambda < 1$: $\mathbf{v}$ is compressed.
  - If $\lambda < 0$: $\mathbf{v}$ is flipped in the opposite direction.
  - If $\lambda = 0$: $\mathbf{v}$ is collapsed into the origin (lies in the null space of $\mathbf{A}$).

3.3 How to Find Eigenvalues: The Characteristic Equation

To solve $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ algebraically:

$$\mathbf{A}\mathbf{v} - \lambda\mathbf{v} = \mathbf{0} \tag{5}$$

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \tag{6}$$

where $\mathbf{I}$ is the $D \times D$ identity matrix.

Why the Determinant Must Equal Zero

If the matrix $(\mathbf{A} - \lambda\mathbf{I})$ were invertible (non-singular), we could multiply both sides by its inverse:

$$\mathbf{v} = (\mathbf{A} - \lambda\mathbf{I})^{-1}\mathbf{0} = \mathbf{0}$$

This would yield only the trivial solution $\mathbf{v} = \mathbf{0}$. But by definition, an eigenvector **must be non-zero**!
Therefore, for non-trivial solutions $\mathbf{v} \neq \mathbf{0}$ to exist, the matrix $(\mathbf{A} - \lambda\mathbf{I})$ **must be singular** (non-invertible). A matrix is singular if and only if its determinant is zero:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{7}$$

This determinant equation expands into a degree-D polynomial in λ, known as the **Characteristic Polynomial**. The roots of this polynomial are the **eigenvalues** $\lambda_1, \lambda_2, \ldots, \lambda_D$.

3.4 Why the Covariance Matrix Guarantees Perfect Properties

In PCA, we do not perform eigendecomposition on an arbitrary matrix; we perform it specifically on the **sample covariance matrix** $\mathbf{C} = \frac{1}{N-1}\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}$.

Three Crucial Properties of the Covariance Matrix

1. Symmetry ($\mathbf{C} = \mathbf{C}^\top$):

$$\mathbf{C}^\top = \left(\frac{1}{N-1}\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}\right)^\top = \frac{1}{N-1}\tilde{\mathbf{Z}}^\top(\tilde{\mathbf{Z}}^\top)^\top = \frac{1}{N-1}\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}} = \mathbf{C}$$

By the **Spectral Theorem** of linear algebra, any real symmetric matrix has:

- Exclusively **real eigenvalues** (no imaginary numbers).
- A complete set of D **mutually orthogonal eigenvectors**:

$$\mathbf{v}_i \cdot \mathbf{v}_j = 0 \quad \text{for all } i \neq j$$

2. Positive Semi-Definiteness ($\mathbf{C} \succeq 0$): For any non-zero vector $\mathbf{u} \in \mathbb{R}^D$:

$$\mathbf{u}^\top\mathbf{C}\mathbf{u} = \mathbf{u}^\top\left(\frac{1}{N-1}\tilde{\mathbf{Z}}^\top\tilde{\mathbf{Z}}\right)\mathbf{u} = \frac{1}{N-1}(\tilde{\mathbf{Z}}\mathbf{u})^\top(\tilde{\mathbf{Z}}\mathbf{u}) = \frac{1}{N-1}\|\tilde{\mathbf{Z}}\mathbf{u}\|^2 \geq 0$$

Since the squared Euclidean norm is strictly non-negative, all eigenvalues are guaranteed to be non-negative:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$$

No negative variances exist!

3. Spectral Decomposition: Because eigenvectors can be chosen orthonormal ($\mathbf{V}^\top \mathbf{V} = \mathbf{I}$), $\mathbf{C}$ decomposes cleanly as:

$$\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$$

where $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_D)$ is a diagonal matrix containing the eigenvalues, and $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_D]$ is an orthogonal matrix whose columns are the eigenvectors.

4 Principal Component Analysis (PCA) Step-by-Step

4.1 The Core Optimization Goal of PCA

What is PCA actually trying to do?

- Suppose we have a standardized cloud of data points $\tilde{\mathbf{Z}} \in \mathbb{R}^{N \times D}$.
- We want to find a unit direction vector $\mathbf{v} \in \mathbb{R}^D$ ($\|\mathbf{v}\| = 1$) such that when we project all data points onto $\mathbf{v}$, the **variance of the projected points is maximized**.

The projection of sample $\tilde{\mathbf{z}}_i$ onto unit vector $\mathbf{v}$ is the scalar $p_i = \tilde{\mathbf{z}}_i \mathbf{v}$. Since the data is zero-mean, the sample variance of the projections is:

$$\text{Var}(p) = \frac{1}{N-1} \sum_{i=1}^{N} (\tilde{\mathbf{z}}_i \mathbf{v})^2 = \frac{1}{N-1} (\tilde{\mathbf{Z}}\mathbf{v})^\top (\tilde{\mathbf{Z}}\mathbf{v}) = \mathbf{v}^\top \left(\frac{1}{N-1} \tilde{\mathbf{Z}}^\top \tilde{\mathbf{Z}} \right) \mathbf{v} = \mathbf{v}^\top \mathbf{C} \mathbf{v}$$

We formulate the constrained optimization problem using Lagrange multipliers:

$$\max_{\mathbf{v}} \mathcal{L}(\mathbf{v}, \lambda) = \mathbf{v}^\top \mathbf{C} \mathbf{v} - \lambda(\mathbf{v}^\top \mathbf{v} - 1)$$

Taking the derivative with respect to $\mathbf{v}$ and setting it to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}} = 2\mathbf{C}\mathbf{v} - 2\lambda\mathbf{v} = \mathbf{0} \implies \mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

Profitable Insight: Variance Equals the Eigenvalue

Notice what happens when we substitute $\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$ back into the variance equation:

$$\text{Variance} = \mathbf{v}^\top \mathbf{C} \mathbf{v} = \mathbf{v}^\top (\lambda \mathbf{v}) = \lambda (\mathbf{v}^\top \mathbf{v}) = \lambda$$

Conclusion:

- The direction of maximum variance is the **eigenvector** $\mathbf{v}_1$ corresponding to the largest eigenvalue λ_1.
- The amount of variance captured along that direction is **exactly equal to the eigenvalue** λ_1!
- The second principal component $\mathbf{v}_2$ is the eigenvector corresponding to λ_2 (orthogonal to $\mathbf{v}_1$), capturing variance λ_2, and so on.

4.2 The Complete 5-Step PCA Algorithm

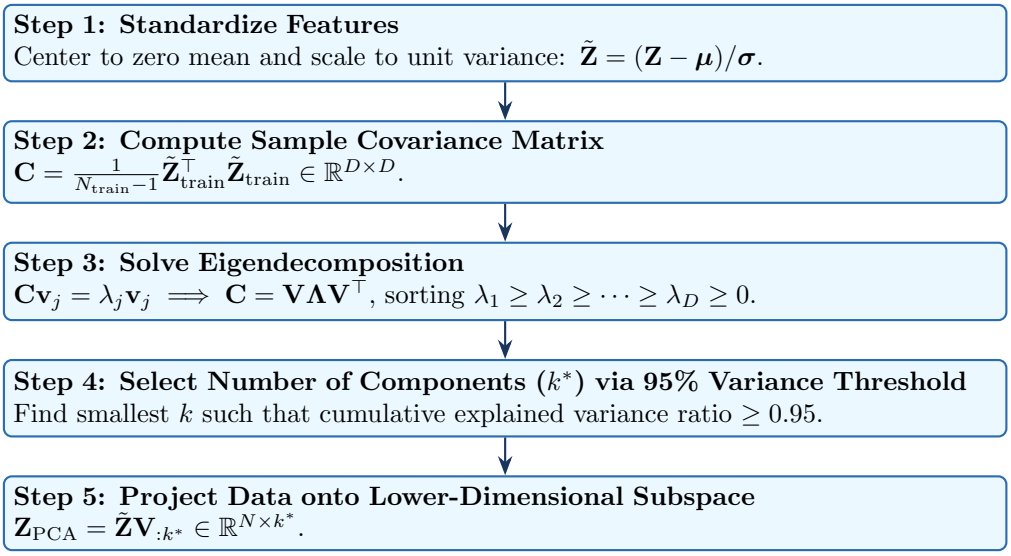


Figure 3: The 5 core algorithmic steps of Principal Component Analysis.

4.3 The 95% Variance Retention Rule (Eq. 3.20)

The total variance present across all D standardized features is:

$$\text{Total Variance} = \sum_{j=1}^{D} \text{Var}(\tilde{\mathbf{z}}_j) = \text{Tr}(\mathbf{C}) = \sum_{j=1}^{D} \lambda_j = D$$

The **Explained Variance Ratio (EVR)** of component j is:

$$\text{EVR}_j = \frac{\lambda_j}{\sum_{i=1}^{D} \lambda_i}$$

The cumulative explained variance retained by keeping the first k components is:

$$\text{CEVR}_k = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{i=1}^{D} \lambda_i}$$

95% Variance Cutoff Formula (Eq. 3.20 in Thesis)

$$k^* = \arg\min_{k} \left(\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{i=1}^{D} \lambda_i} \geq 0.95 \right)$$

We retain exactly the top k^* eigenvectors $[\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_{k^*}] = \mathbf{V}_{:k^*} \in \mathbb{R}^{D\times k^*}$, discarding the remaining $D - k^*$ trailing dimensions.

4.4 Dimensionality Reduction Results in the Thesis

Table 1: Empirical PCA Dimensionality Reduction Results (95% Variance Threshold).

Dataset & Model	Original Dim (D)	Retained Dim (k^*)	Dimensionality Reduction
FineGym (4 classes)			
V-JEPA 2 (ViT-H)	1,280	**90**	−93.0%
Qwen2.5-VL (7B)	3,584	**142**	−96.0%
HMDB51 (51 classes)			
V-JEPA 2 (ViT-H)	1,280	**286**	−77.7%
Qwen2.5-VL (7B)	3,584	**719**	−79.9%

Why did HMDB51 retain more dimensions than FineGym?

Defense Question: "Why did PCA retain 286 dimensions for V-JEPA 2 on HMDB51, but only 90 dimensions on FineGym?"
Examiner-Ready Answer:

- **Intrinsic Semantic Complexity:** FineGym evaluates 4 highly structured gymnastic apparatus events (Vault, Uneven Bars, Balance Beam, Floor Exercise). The visual discriminators are tightly clustered around specific apparatus geometry and athlete body orientation, requiring very few principal axes to capture 95% of feature variation.
- **Broad Spectrum of Human Actions:** HMDB51 spans 51 diverse daily, athletic, and combat actions (e.g., *brush hair, catch, climb, draw sword, drink, ride horse, shoot bow, sword*). Distinguishing across 51 diverse semantic clusters requires an intrinsically higher-dimensional manifold ($k^* = 286$ and 719) to preserve subtle motion and object interactions.

5 Step-by-Step Worked Numerical Walkthrough

To solidify complete intuition, let us walk through a complete numerical calculation by hand on a 2-dimensional dataset ($N = 4$ samples, $D = 2$ features).

5.1 Initial Data Matrix

Suppose our extracted feature matrix $\mathbf{Z} \in \mathbb{R}^{4\times 2}$ consists of 4 video clip embeddings:

$$\mathbf{Z} = \begin{bmatrix} 2 & 4 \\ 4 & 6 \\ 6 & 8 \\ 8 & 14 \end{bmatrix}$$

Here, column 1 is Feature 1 (z_1) and column 2 is Feature 2 (z_2).

5.2 Step 1: Compute Means and Standard Deviations

For Feature 1 ($j = 1$):

$$\mu_1 = \frac{2+4+6+8}{4} = \frac{20}{4} = 5.0$$

$$\text{Var}_1 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4-1} = \frac{(-3)^2 + (-1)^2 + (1)^2 + (3)^2}{3} = \frac{9+1+1+9}{3} = \frac{20}{3} \approx 6.667$$

$$\sigma_1 = \sqrt{\frac{20}{3}} \approx 2.582$$

For Feature 2 ($j = 2$):

$$\mu_2 = \frac{4+6+8+14}{4} = \frac{32}{4} = 8.0$$

$$\text{Var}_2 = \frac{(4-8)^2 + (6-8)^2 + (8-8)^2 + (14-8)^2}{4-1} = \frac{(-4)^2 + (-2)^2 + (0)^2 + (6)^2}{3} = \frac{16+4+0+36}{3} = \frac{56}{3} \approx 18$$

$$\sigma_2 = \sqrt{\frac{56}{3}} \approx 4.320$$

Notice how Feature 2 has nearly **three times the variance** of Feature 1 in raw units (18.667 vs 6.667). Without StandardScaler, Feature 2 would artificially dominate the analysis!

5.3 Step 2: Construct Standardized Matrix $\tilde{\mathbf{Z}}$

Using $\tilde{z}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j}$:

$$\tilde{\mathbf{Z}} = \begin{bmatrix} \frac{2-5}{2.582} & \frac{4-8}{4.320} \\ \frac{4-5}{2.582} & \frac{6-8}{4.320} \\ \frac{6-5}{2.582} & \frac{8-8}{4.320} \\ \frac{8-5}{2.582} & \frac{14-8}{4.320} \end{bmatrix} \approx \begin{bmatrix} -1.162 & -0.926 \\ -0.387 & -0.463 \\ +0.387 & 0.000 \\ +1.162 & +1.389 \end{bmatrix}$$

5.4 Step 3: Compute the Covariance Matrix C

The sample covariance matrix is:

$$\mathbf{C} = \frac{1}{N-1}\tilde{\mathbf{Z}}^\top \tilde{\mathbf{Z}} = \frac{1}{3}\begin{bmatrix} \sum \tilde{z}_{i1}^2 & \sum \tilde{z}_{i1}\tilde{z}_{i2} \\ \sum \tilde{z}_{i1}\tilde{z}_{i2} & \sum \tilde{z}_{i2}^2 \end{bmatrix}$$

Because each column was standardized with sample standard deviation ($N-1$ divisor), the diagonal elements are strictly equal to 1.0:

$$C_{11} = 1.0, \quad C_{22} = 1.0$$

Now we compute the cross-correlation $r = C_{12} = C_{21}$:

$$\begin{aligned} \sum_{i=1}^{4} \tilde{z}_{i1}\tilde{z}_{i2} &= (-1.162)(-0.926) + (-0.387)(-0.463) + (0.387)(0.000) + (1.162)(1.389) \\ &= 1.076 + 0.179 + 0.000 + 1.614 = 2.869 \\ C_{12} &= \frac{2.869}{3} \approx 0.956 \end{aligned}$$

Thus, our covariance matrix is:

$$\mathbf{C} = \begin{bmatrix} 1.000 & 0.956 \\ 0.956 & 1.000 \end{bmatrix}$$

Notice that $\mathbf{C}$ is symmetric ($C_{12} = C_{21}$) and exhibits high correlation ($r = 0.956$) between the two features.

5.5 Step 4: Solve the Characteristic Equation for Eigenvalues

We solve $\det(\mathbf{C} - \lambda\mathbf{I}) = 0$:

$$\det \begin{bmatrix} 1-\lambda & 0.956 \\ 0.956 & 1-\lambda \end{bmatrix} = 0$$

Evaluating the 2×2 determinant:

$$(1-\lambda)(1-\lambda) - (0.956)^2 = 0$$

$$(1-\lambda)^2 - 0.914 = 0$$

$$(1-\lambda)^2 = 0.914 \implies 1-\lambda = \pm\sqrt{0.914} = \pm 0.956$$

Solving for λ:

$$\lambda_1 = 1 + 0.956 = \mathbf{1.956}$$

$$\lambda_2 = 1 - 0.956 = \mathbf{0.044}$$

Verification of Eigenvalue Properties

- **Trace Property Check:** $\mathrm{Tr}(\mathbf{C}) = C_{11} + C_{22} = 1 + 1 = 2.0$.

$$\lambda_1 + \lambda_2 = 1.956 + 0.044 = 2.000 \quad (\textbf{Verified!})$$

- **Determinant Property Check:** $\det(\mathbf{C}) = (1)(1) - (0.956)^2 = 1 - 0.914 = 0.086$.

$$\lambda_1 \times \lambda_2 = (1.956)(0.044) = 0.086 \quad (\textbf{Verified!})$$

- **Explained Variance Ratios:**

$$\text{EVR}_1 = \frac{1.956}{2.000} = 97.8\%, \qquad \text{EVR}_2 = \frac{0.044}{2.000} = 2.2\%$$

5.6 Step 5: Solve for the Eigenvectors

Finding $\mathbf{v}_1$ for $\lambda_1 = 1.956$:

$$(\mathbf{C} - \lambda_1 \mathbf{I})\mathbf{v}_1 = \mathbf{0} \implies \begin{bmatrix} 1 - 1.956 & 0.956 \\ 0.956 & 1 - 1.956 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -0.956 & 0.956 \\ 0.956 & -0.956 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This yields the equation:

$$-0.956 v_{11} + 0.956 v_{12} = 0 \implies v_{11} = v_{12}$$

We require unit length $\|\mathbf{v}_1\|^2 = v_{11}^2 + v_{12}^2 = 1$:

$$v_{11}^2 + v_{11}^2 = 1 \implies 2v_{11}^2 = 1 \implies v_{11} = \frac{1}{\sqrt{2}} \approx 0.7071$$

Thus:

$$\mathbf{v}_1 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

Finding $\mathbf{v}_2$ for $\lambda_2 = 0.044$:

$$(\mathbf{C} - \lambda_2 \mathbf{I})\mathbf{v}_2 = \mathbf{0} \implies \begin{bmatrix} 1 - 0.044 & 0.956 \\ 0.956 & 1 - 0.044 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0.956 & 0.956 \\ 0.956 & 0.956 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies v_{21} = -v_{22}$$

Normalizing to unit length gives:

$$\mathbf{v}_2 = \begin{bmatrix} -0.7071 \\ 0.7071 \end{bmatrix}$$

Orthogonality Check:

$\mathbf{v}_1 \cdot \mathbf{v}_2 = (0.7071)(-0.7071) + (0.7071)(0.7071) = -0.5 + 0.5 = 0.000$ (**Perfect Orthogonality!**)

5.7 Step 6: Apply 95% Cutoff and Project Data

Since $\text{EVR}_1 = 97.8\% \geq 95\%$, the 95% variance threshold retains **only** $k^* = 1$ **component**! We reduce the data from 2 dimensions to 1 dimension using projection matrix $\mathbf{V}_{:1} = \mathbf{v}_1$:

$$\mathbf{Z}_{\text{PCA}} = \tilde{\mathbf{Z}}\mathbf{v}_1 = \begin{bmatrix} -1.162 & -0.926 \\ -0.387 & -0.463 \\ +0.387 & 0.000 \\ +1.162 & +1.389 \end{bmatrix} \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix} = \begin{bmatrix} (-1.162 + -0.926) \times 0.7071 \\ (-0.387 + -0.463) \times 0.7071 \\ (0.387 + 0.000) \times 0.7071 \\ (1.162 + 1.389) \times 0.7071 \end{bmatrix} = \begin{bmatrix} \mathbf{-1.476} \\ \mathbf{-0.601} \\ \mathbf{+0.274} \\ \mathbf{+1.804} \end{bmatrix}$$

Verifying the Variance of the Projected Data

Let us check the sample variance of these 1D projected scores:

$$\text{Mean} = \frac{-1.476 - 0.601 + 0.274 + 1.804}{4} = 0.000$$

$$\text{Var}(z_{\text{PCA}}) = \frac{(-1.476)^2 + (-0.601)^2 + (0.274)^2 + (1.804)^2}{3} = \frac{2.179 + 0.361 + 0.075 + 3.254}{3} = \frac{5.869}{3} = \mathbf{1.95}$$

The variance of the projected coordinates exactly equals $\lambda_1 = 1.956$! We preserved 97.8% of the total information while discarding 50% of the dimensions.

6 Architectural Rationale: Why Baseline Classifiers Use PCA but HAF-STD Net Bypasses It

One of the most critical methodological distinctions in the thesis is:

- **Path 1 (Baseline Classifiers):** Standardized features $\longrightarrow$ **PCA** (95% variance) $\longrightarrow$ Classifiers.
- **Path 2 (HAF-STD Net):** Standardized multi-layer temporal features $\longrightarrow$ **NO PCA (Bypassed)** $\longrightarrow$ HAF-STD Net.

6.1 Why Baseline Classifiers Strictly Require PCA

1. **Mitigating the Curse of Dimensionality ($N \ll D$):**
   - On the FineGym dataset, the total available sample size is $N = 272$ video clips.
   - The raw feature dimension from V-JEPA 2 is $D = 1{,}280$, and from Qwen2.5-VL is $D = 3{,}584$.
   - In both cases, $N \ll D$ ($272 < 1280$ and $272 < 3584$). When the number of features exceeds the number of training samples, linear models and shallow MLPs have infinite degrees of freedom to memorize spurious noise patterns rather than generalizable action dynamics.
   - PCA projects the dimensionality down to $k^* = 90$ (V-JEPA) and $k^* = 142$ (Qwen), restoring a favorable sample-to-feature ratio ($N > k^*$).
2. **Eliminating Multicollinearity:**
   - Over-parameterized vision transformers produce highly correlated feature channels across visual tokens.
   - Multicollinear features destabilize classical classifiers:
     - In **Logistic Regression**, correlated features cause gradient instability and explode weight norms.
     - In **Support Vector Machines (SVM-RBF)**, distance metrics become distorted because redundant directions are counted multiple times in Euclidean distance computations.
   - PCA projects features into an orthogonal coordinate system where all off-diagonal covariances are identically zero ($C_{ij} = 0$ for $i \neq j$), providing perfectly decorrelated inputs.
3. **Computational Scalability:**
   - Training an SVM with an RBF kernel scales as $O(N^2 \cdot D)$ in memory and between $O(N^2 \cdot D)$ to $O(N^3 \cdot D)$ in time.
   - Random Forests require evaluating split thresholds across all candidate features at every tree node ($O(N_{\text{trees}} \cdot \sqrt{D} \cdot N \log N)$).
   - Reducing $D = 3{,}584$ to $k^* = 142$ yields a 25$\times$ speedup during hyperparameter cross-validation and training.

6.2 Why HAF-STD Net Deliberately Bypasses PCA

Core Architectural Defense: Why HAF-STD Net Must Bypass PCA

Defense Question: "Why did you apply PCA to your baseline classifiers, but bypass PCA for your proposed HAF-STD Net? Isn't that an unfair or inconsistent comparison?"
Examiner-Ready Answer: "Bypassing PCA for HAF-STD Net is not an inconsistency; it is a fundamental architectural requirement dictated by the physical nature of temporal signal processing:

1. **Destruction of Spatiotemporal Topology:** PCA requires flattening multidimensional data into a 2D matrix $\mathbf{Z} \in \mathbb{R}^{N \times D_{\text{total}}}$. Doing so treats the temporal axis ($T = 8$) and discrete layer depth ($K = 4$) as an arbitrary bag of unordered numbers, destroying their temporal sequence and layer hierarchy.

2. **Requirement of Continuous Time for Haar DWT:** HAF-STD Net introduces **Spectral-Temporal Decomposition via Haar Discrete Wavelet Transforms**. Wavelets operate on ordered, continuous time series:

$$\mathbf{a}_k = \frac{1}{2}(\mathbf{h}_{2t} + \mathbf{h}_{2t+1}), \qquad \mathbf{d}_k = \frac{1}{2}(\mathbf{h}_{2t} - \mathbf{h}_{2t+1})$$

   Haar DWT decomposes high-frequency motion transients from low-frequency body postures. If features had been compressed via PCA, the temporal phase relationships required to compute velocity and acceleration would be destroyed.

3. **Internal Dimensionality Reduction via Learned Attention:** HAF-STD Net does not suffer from the curse of dimensionality because it incorporates its own **12-head cross-layer attention mechanism** and bottleneck projection layers (with 4.8M parameters). It learns *task-specific, non-linear dimensionality reduction* optimized directly by classification loss and label smoothing, which is strictly superior to unsupervised linear variance compression."

7 Quick Reference & Examination Defense Cheat Sheet

Table 2: Comprehensive Formula Summary for Preprocessing and PCA.

Concept	Mathematical Formula	Practical Role
Frozen Forward Pass	$\mathbf{z}_i = f(\mathbf{x}_i; \boldsymbol{\Theta}_{\text{frozen}}),\ \nabla_{\boldsymbol{\Theta}}\mathcal{L} = \mathbf{0}$	Isolates pretrained representation quality without backbone tuning.
Feature Standardization	$\tilde{z}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j}$	Prevents high-scale dimensions from dominating PCA variance.
Sample Covariance	$\mathbf{C} = \frac{1}{N_{\text{train}}-1}\tilde{\mathbf{Z}}_{\text{train}}^{\top}\tilde{\mathbf{Z}}_{\text{train}}$	Quantifies pairwise linear correlations between standardized features.
Eigen-Equation	$\mathbf{C}\mathbf{v}_j = \lambda_j \mathbf{v}_j$	Defines invariant projection axes (eigenvectors) and variances (eigenvalues).
Characteristic Equation	$\det(\mathbf{C} - \lambda\mathbf{I}) = 0$	Yields degree-D polynomial to calculate eigenvalues λ_j.
95% Variance Rule	$k^* = \arg\min_k \left(\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{i=1}^{D} \lambda_i} \geq 0.95 \right)$	Selects minimum number of components preserving 95% total variance.
Subspace Projection	$\mathbf{Z}_{\text{PCA}} = \tilde{\mathbf{Z}}\mathbf{V}_{:k^*} \in \mathbb{R}^{N \times k^*}$	Projects standardized features into decorrelated k^*-dimensional subspace.

7.1 Top 5 Defense Questions & Bulletproof Answers

Q1: What does an eigenvector physically represent in PCA?

Answer: An eigenvector represents an orthogonal direction in the high-dimensional feature space along which the data varies. The first eigenvector points in the direction of greatest variance; each subsequent eigenvector points in the direction of greatest remaining variance orthogonal to all preceding components.

Q2: What does an eigenvalue physically represent in PCA?

Answer: An eigenvalue λ_j is exactly equal to the numerical variance of the data points when projected onto the corresponding eigenvector $\mathbf{v}_j$. The sum of all eigenvalues equals the total variance of the dataset (D for standardized data).

Q3: Why must StandardScaler be fitted ONLY on the training split?

Answer: To prevent data leakage. If test set samples are used to compute the mean and standard deviation, information about the evaluation distribution leaks into the model training pipeline, creating overly optimistic, invalid performance metrics.

Q4: What is the difference between Float16 and NF4 quantization?

Answer: Float16 is a 16-bit floating-point format (1 sign, 5 exponent, 10 mantissa bits) that halves memory footprint while retaining dynamic range for standard GPU tensor core operations. NF4 is a 4-bit quantile-based quantization format specifically optimized

for normally distributed neural network weights, reducing Qwen2.5-VL from 14 GB to ≈ 3.9 GB VRAM without significant degradation.

Q5: What is the decorrelation property of PCA?

Answer: When features are projected onto the principal components, the resulting covariance matrix of the projected features is strictly diagonal:

$$\text{Cov}(\mathbf{Z}_{\text{PCA}}) = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_{k^*})$$

All off-diagonal entries are 0, meaning that every projected dimension is completely linearly uncorrelated with every other dimension.